INDEX

NAME Shivani R.J SUBJECT Computer Networks

STD. _____ DIV. _____ ROLL NO. 240701500 SCHOOL _____

SR. NO.	Date ~~PAGE NO.~~	TITLE	DATE	TEACHER'S SIGN REMARKS
1.	24.07.26	Practical - 1 Study of various Network Commands		24/7/26 9
2.	28.07.26	Practical - 2 Study of Network Cables		9
3.	10.08.26	Practical - 3 Experiments on Cisco Packet Tracer		9
4.	16.08.26	Practical - 4 Setup and configure a LAN using a switch & Ethernet Cables	9	18/8
5.	8.8.26	Practical - 6 Error Detection at Data Link Layer	9	9/9/26
6.	9.9.26	Practical - 5 Experiments on Packet capture tool : Wireshark		9/9/26 9
7.	29.9.26	Practical - 7 Flow control at Data Link Layer	9	29/9/26
8.	30.9.26	Practical - 8 NMAP to discover Live Hosts using Nmap Scans on TryHackMe Platform	9	30/9/26
9.	30.9.26	Practical - 9 Implementation of Subnetting in Cisco Packet Tracer simulator		24/9 9

Scanned with OKEN Scanner

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
10.	30.9.26	Practical - 12 End - End Communication at Transport Layer		
11.	2.10.26	Implement Ping Program		
12.	2.10.26	Write a code using RAW socket to implement packet sniffing		
13.	5.10.26	Internetwork using Router		
14	10.10.26	RIP routing		

Scanned with OKEN Scanner

Exp. No: 1
24.07.26

Study of various Network commands used in Linux and Windows

Aim:

Study of various Network commands used in Linux and Windows.

Basic Networking commands

1. arp -a

Displays the IP address and MAC address of devices connected to your network.

2. hostname

Shows the name of your computer.

3. ipconfig /all

Displays complete network information like IP address, Gateway, DNS and MAC address.

4. nbtstat -a

Displays NetBIOS information of a computer. Used to troubleshoot Windows network issues.

Scanned with OKEN Scanner

5. netstat

Displays active network connections, listening ports, and routing information.

netstat -n

6. nslookup

Finds the IP address of a website using DNS.

nslookup www.google.com

7. pathping

Combines Ping + Traceroute. Shows the path and packet loss at each router.

8. ping

Checks whether another computer or website is reachable.

ping google.com

9. Route

used to show / manipulate the IP routing table.

route print

Scanned with OKEN Scanner

Linux Networking Commands

1. ip

ip command is one of the basic commands every administrator will need in daily work, from setting up new systems and assigning IPs to troubleshooting existing systems.

ip <Options> <Object> <Command>

a) To show the IP addresses assigned to an interface on your server

ip address show

b) To assign an IP to an interface

sudo ip address add 192.168.1.254/24 dev enp0s3

c) to delete an IP on an interface

sudo ip address del 192.168.1.254/24 dev enp0s3

d) Alter the status of the interface by bringing the interface enp0s3

sudo ip link set enp0s3 up

e) Alter the status of the interface by bringing the interface enp0s3

sudo ip link set enp0s3 down

f) Alter the status of the interface by enabling promiscuous mode for.

sudo ip link set enp0s3 promisc on

Scanned with OKEN Scanner

g. Add a default route via the local gateway 192.168.1.254 that can be reached on device enp0s3

sudo ip route add default via 192.168.1.254 dev enp0s3

h. Display the route taken for IP 10.10.1.4

ip route get

2. ifconfig

displays or configures network interfaces (older Linux command).

3. mtr

Network troubleshooting tool that combines Ping and Traceroute.

mtr google.com

mtr <options> hostname / IP

a. Basic mtr command

mtr google.com

b. Show numeric IP addresses

mtr -g google.com

c. Show numeric IP addresses and hostnames

mtr -b google.com

d. Set the number of pings that you want to

4. tcpdump

captures and displays network packets for analysis

tcpdump -i enp0s3

5. ping

Test connectivity between two devices

ping google.com

ping -c 5 google.com

Configuring an Ethernet connection by using nmcli

nmcli (Network Manager Command Line Interface) is a Linux command used to view, configure and manage network connections from the terminal

Procedure

1. List all network connections

nmcli connection show

2. Create a new Ethernet connection

nmcli connection add con-name <connection-name> ifname <device-name> type ethernet.

3. Rename the connection

nmcli connection modify "Wired connection-1"

Scanned with OKEN Scanner

4. View current settings

nmcli connection show

5. Configure the IPv4 settings

6. Activate the profile.

i) Install on RHEL / Fedora / centos distributions

dnf install -y tcpdump

ii) List all available interfaces for capture

tcpdump -D

iii) Capture live packets on a specific interface (eth0)

tcpdump -i eth0

Result

Thus the output for various windows and Linux commands has been verified.

12/12/24

Scanned with OKEN Scanner

Exp. No 2
28.07.26

Study of Network cables

Aim:

To study of different types of Network cables.

a) Understand different types of network cable

Different type of cables used in networking are:

1. Unshielded Twisted Pair (UTP) Cable
2. Shielded Twisted Pair (STP) Cable
3. Coaxial Cable
4. Fibre Optic Cable

Cable Type	Category	Maximum Data Transmission	Advantages/ Disadvantages	Application/ Use
UTP	Category 3	10 Mbps	**Advantages** * Cheaper in cost. * Easy to install as they have a smaller overall diameter **Disadvantages** * More prone to (EMI) Electromagnetic interference &	10 Base - T Ethernet
	Category 5	Up to 100 Mbps		Fast Ethernet, Gigabit Ethernet
	Category 5e	1 Gbps		Fast Ethernet, Gigabit Ethernet

STP	Category 6, 6a	10 Gbps	Advantages *Shielded *Faster than UTP. *Less susceptible to noise and interference. Disadvantages *Expensive *Greater installation effort	Gigabit Ethernet, 10G Ethernet (55m) Widely used in data centres
SSTP	Category 7	10 Gbps		Gigabit Ethernet, 10G Ethernet (100m)
Coaxial cable	RG-6 RG-59 RG-11	10-100 Mbps	Advantages *High bandwidth *Immune to interference *Low loss bandwidth *Versatile Disadvantages *Limited distance *Cost *Size is bulky	Speed of signal is 500m Television network High speed internet connections
Fibre optics cable	Single mode Multi mode	100 Gbps	Advantages *High speed *High bandwidth *High security *Long distance Disadvantages *Expensive *Required skilled	Maximum distance of fibre optics cable is around 100 meters

Result

Networ

4/8/26

Scanned with OKEN Scanner

Result:

Thus the study of different types of Network cables is done successfully.

4/8/26

Scanned with OKEN Scanner

Exp No: 3
10.8.26

Experiments on CISCO PACKET TRACER

(Simulation Tool)

Aim:

To study the Packet Tracer tool Installation and User Interface Overview.

Introduction:

Cisco Packet Tracer is a network simulation tool used to create, configure and troubleshoot computer networks without using real hardware.

It helps students understand the working of network devices such as PCs, Hubs, Switches and Routers.

Installation

1. Download Cisco Packet Tracer from the Cisco Networking Academy website.
2. Log in with your cisco account.
3. Download the version suitable for your operating system
4. Run the installer and follow the installation instructions.

Packet Tracer User Interface

The main parts of the Packet Tracer are:

1. Menu Bar - Used for opening, saving and managing files.

2. Main Toolbar - Contains shortcuts such as Save, Open, Undo, Redo and Zoom

3. Logical / Physical Workspace - Used to switch between logical and physical views.

4. Workspace - Area where the network topology is created.

5. Common Tools Bar - Contains tools for selecting, deleting, inspecting and adding packets.

6. Realtime / Simulation Mode - Used to view the network normally or observe packet movement.

7. Network Component Box - Contains routers, switches, hubs, PCs, and other devices.

8. Packet Box - Used to create and observe test packets.

Procedure

1) Peer-to-Peer Network

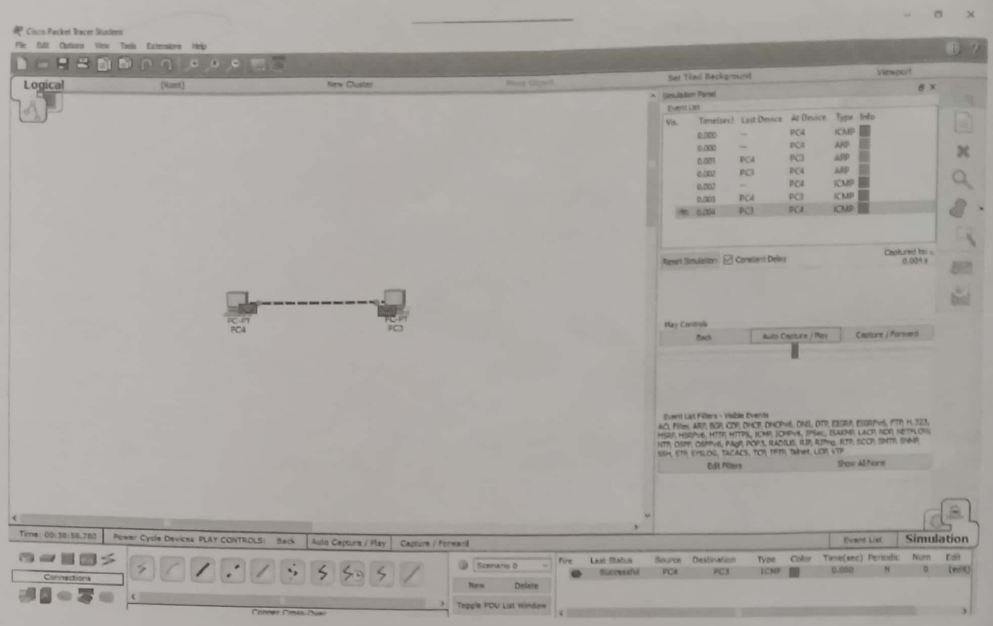
Open Cisco Packet Tracer.

Add 2 PCs to the workspace.

Select connections and choose Copper Cross-Over cable.

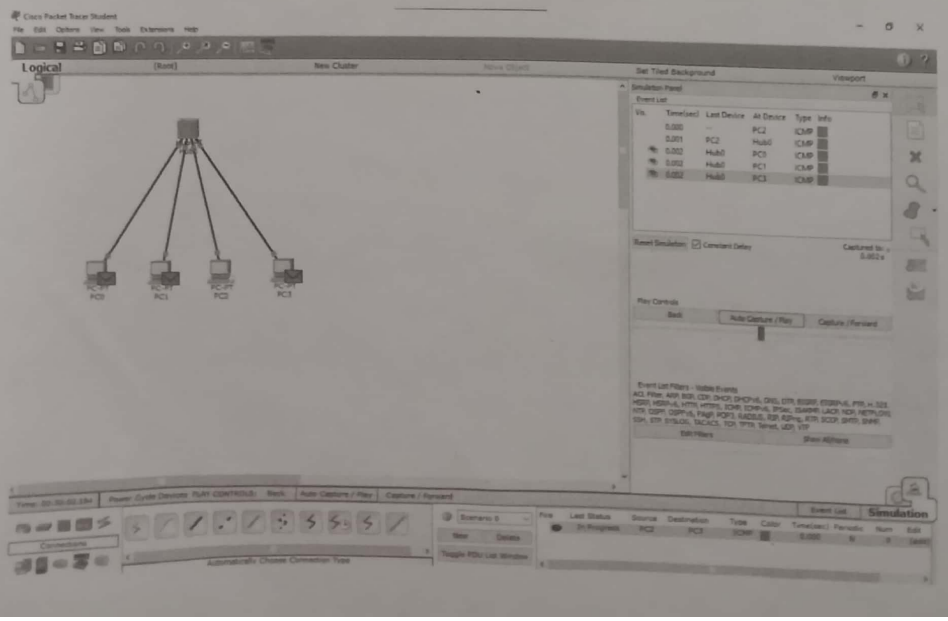
Connect PC0 directly to PC1

Scanned with OKEN Scanner



Network using Hub

1. Open Cisco Packet Tracer.
2. Add 4 PCs and 1 Hub to the workspace.
3. Connect all PCs to the Hub using copper straight-through cables.



Scanned with OKEN Scanner

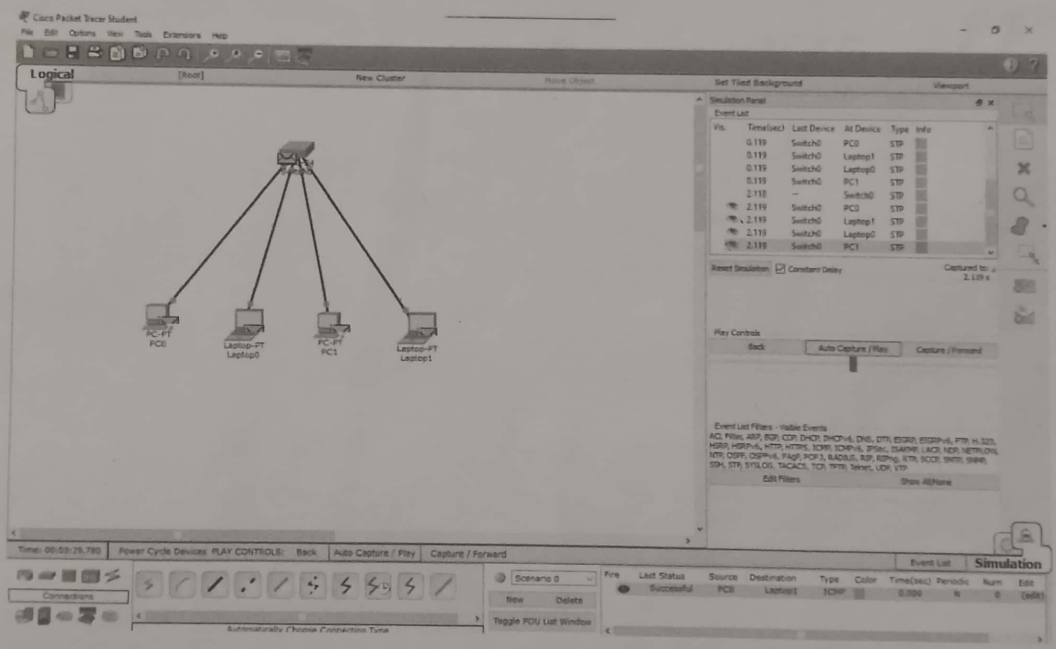
Network using switch

Add 4 PCs and 1 switch

Connect all PCs to the switch using copper straight - through cables.

Configure IP addresses in the same network.

Send a PDU from one PC to another.



Scanned with OKEN Scanner

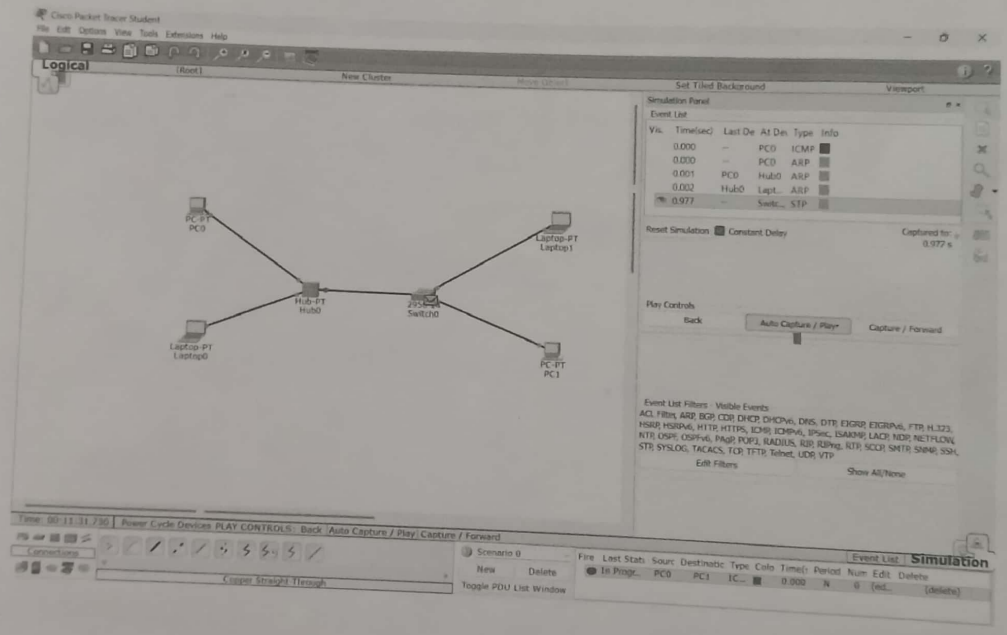
4) Network using Both Hub and Switch

Open Cisco Packet Tracer.

Add 4 PCs, 1 Hub and 1 Switch to the workspace

Connect PC0 and PC1 to the Hub using Copper Straight-Through cables.

Connect PC2 and PC3 to the switch using a copper straight-through cable.



Result:

Thus the study of Packet Tracer tool is implemented successfully.

8/8/26

Scanned with OKEN Scanner

Ex: NO: 4
Date: 08.08.25

Setup and configure a LAN using a switch and Ethernet cables

Aim:

To set up and configure a LAN (Local Area Network) using a switch and Ethernet cables.

What is LAN?

A LAN is a network that connects computers and other devices within a limited area such as a home, school or office. It allows device to communicate and share resources such as files and printers.

Requirements

4 Computers
1 Network Switch
4 Ethernet cables

Procedure

1. Connect 4 computers to the switch using Ethernet cables.
2. Assign IP addresses to the computers:

PC1 : 10.10.10.1
PC 2 : 10.10.10.2
PC 3 : 10.10.10.3
PC 4 : 10.10.10.4

3. Configure the switch
4. Open command prompt on a computer and use the ping command to check

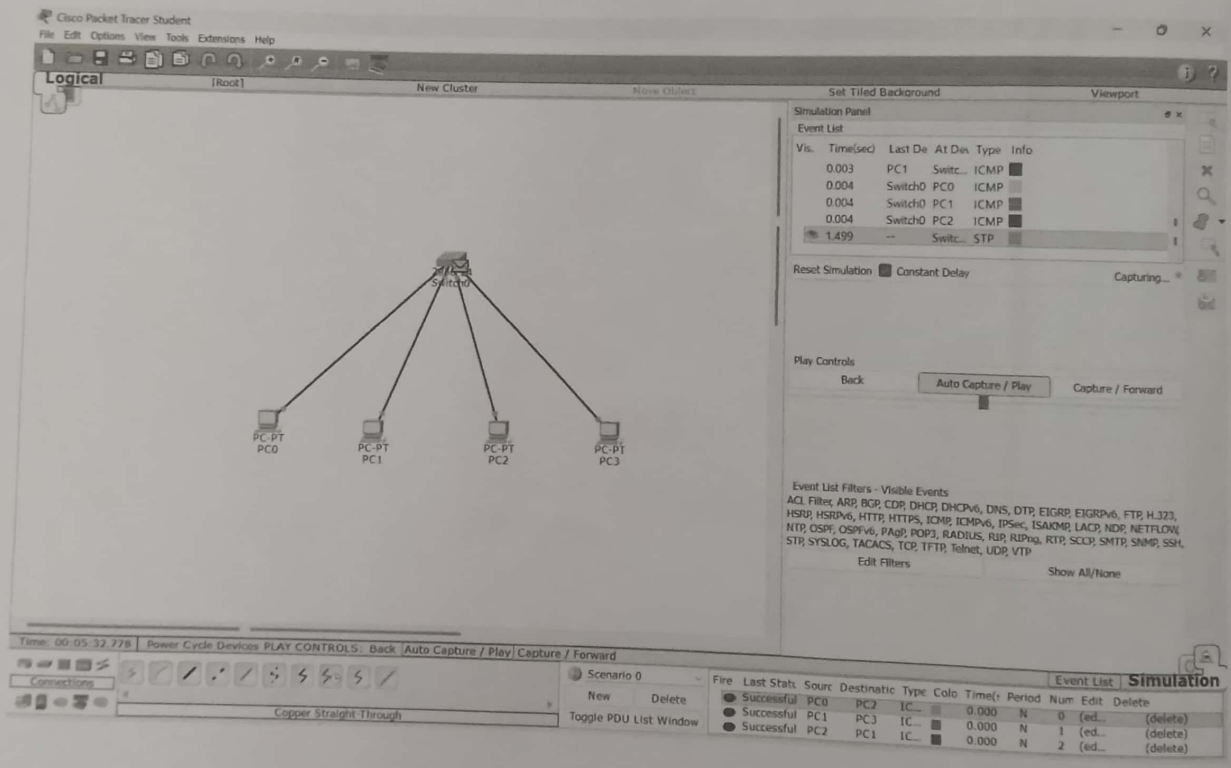
Scanned with OKEN Scanner

connectivity.

5. Select a folder on one computer and enable file sharing for users on the same LAN.

6. Access the shared folder from the other computers on the network.

7. Verify that all computers can communicate and share files successfully.



Result

Thus to setup LAN using switch and ethernet is executed successfully.

Scanned with OKEN Scanner

Ex: No: 6
8.9.26

Error Correction at Data Link Layer

Aim:

To write a program to implement error detection and correction using hamming code.

Code:

```
def calculate_parity_bits(data):
    m = len(data)
    r = 0
    while (2**r) < (m+r+1):
        r += 1
    return r

def encode(data):
    data = list(map(int, data))
    m = len(data)
    r = calculate_parity_bits(data)
    n = m + r
    hamming = [0] * (n+1)
    j = 0
    for i in range(1, n+1):
        if (i & (i-1)) != 0:
            hamming[i] = data[j]
            j += 1
    for p in range(r):
        position = 2**p
```

Scanned with OKEN Scanner

```
        parity = 0
        for i in range (1, n+1):
            if i & position:
                parity ^= hamming[i]
        hamming[position] = parity
    return hamming[1:]

def detect_and_correct(hamming):
    n = len(hamming)
    hamming = [0] + hamming
    error_position = 0
    p = 1
    while p <= n:
        parity = 0
        for i in range(1, n+1):
            if i & p:
                parity ^= hamming[i]
        if parity != 0:
            error_position += p
        p *= 2
    if error_position == 0:
        print("No error detected.")
    else:
        print("Error detected at position:",
              error_position)
```

Scanned with OKEN Scanner

```
        hamming[error_position] ^= 1
        print("Error corrected.")
    return hamming[1:]

data = input("Enter data bits: ")
hamming = encode(data)
print("Hamming Code:", ''.join(map(str, hamming)))
position = int(input("Enter position to introduce
                    error: ").format(len(hamming)))
hamming[position-1] ^= 1
print("Received data with error:", ''.join(map
                    (str, hamming)))
corrected = detect_and_correct(hamming)
print("Corrected Hamming Code:" ''.join(map
                    (str, corrected)))
```

Input

1011

Output

```
Enter data bits: 1011
Hamming Code: 0110011
Enter position to introduce error: 5
Received data with error: 0110111
Error detected at position: 5
Error corrected.
Corrected Hamming Code: 0110011
```

Scanned with OKEN Scanner

Result

Thus a program to implement error detection and correction is executed successfully.

9/9/24

Ex: No: 5

9.9.26

Experiments on Packet capture

tool : Wireshark

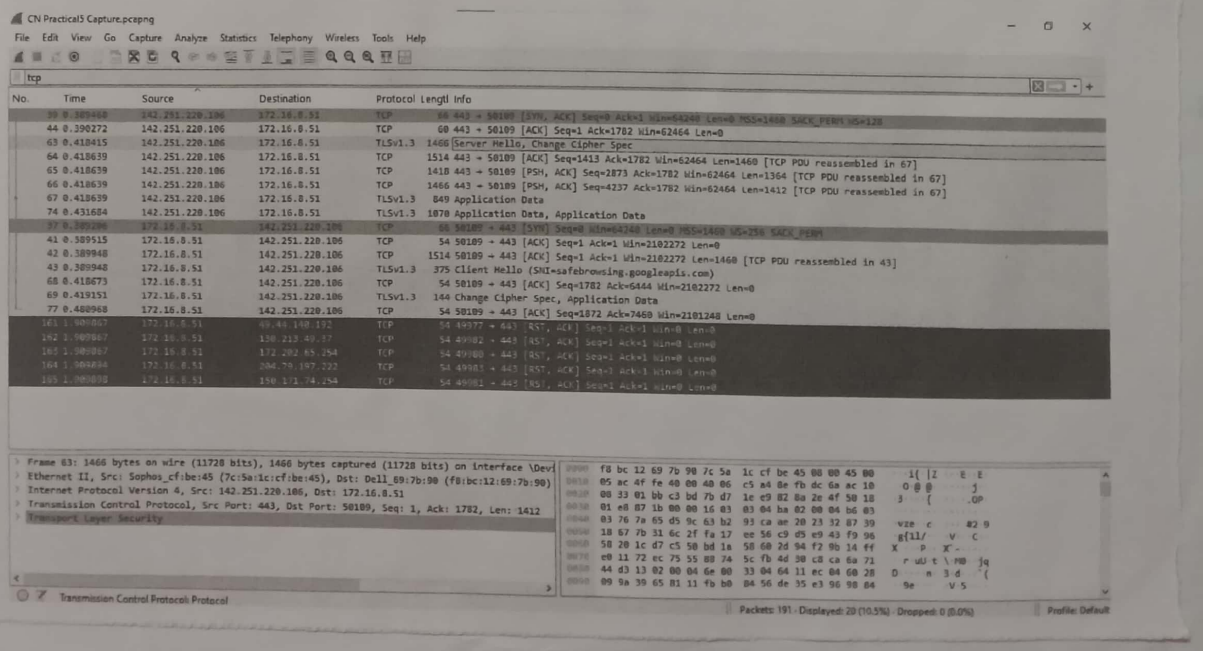
Aim:

To understand the features of wireshark as a packet capture tool and understand encapsulation of information at various layers of a Protocol stack.

Packet Filtering and Analysis

1. TCP/UDP Packets

Enter the following filters in the wireshark filter bar: tcp or udp

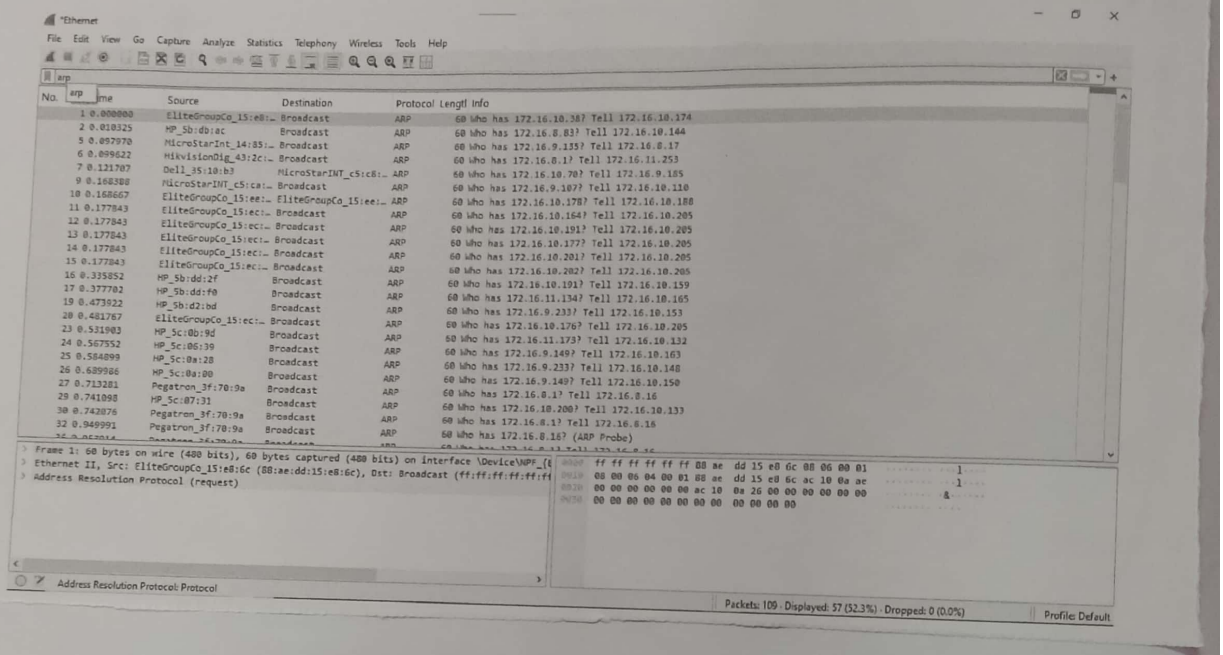


2. ARP Packets

Enter the following filter

arp

The ARP packets can be inspected to observe the communication between devices and the corresponding IP and MAC address information.



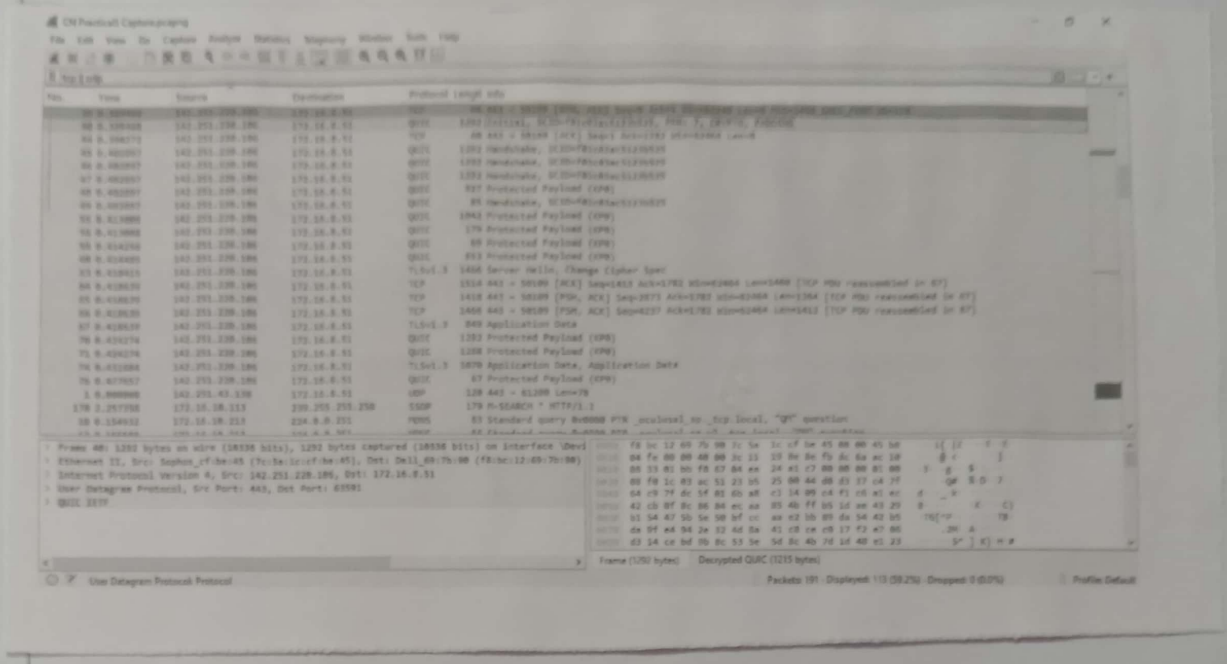
3. DNS Packets

Enter :

dns

The DNS packets show the communication between the computer and the DNS server. The packet details can be inspected to identify DNS queries and responses

Scanned with OKEN Scanner



4. HTTP Packets

Enter :

http

HTTP packets can be selected and inspected to view the packet information, source, destination and HTTP protocol fields

5. IP/ICMP Packets

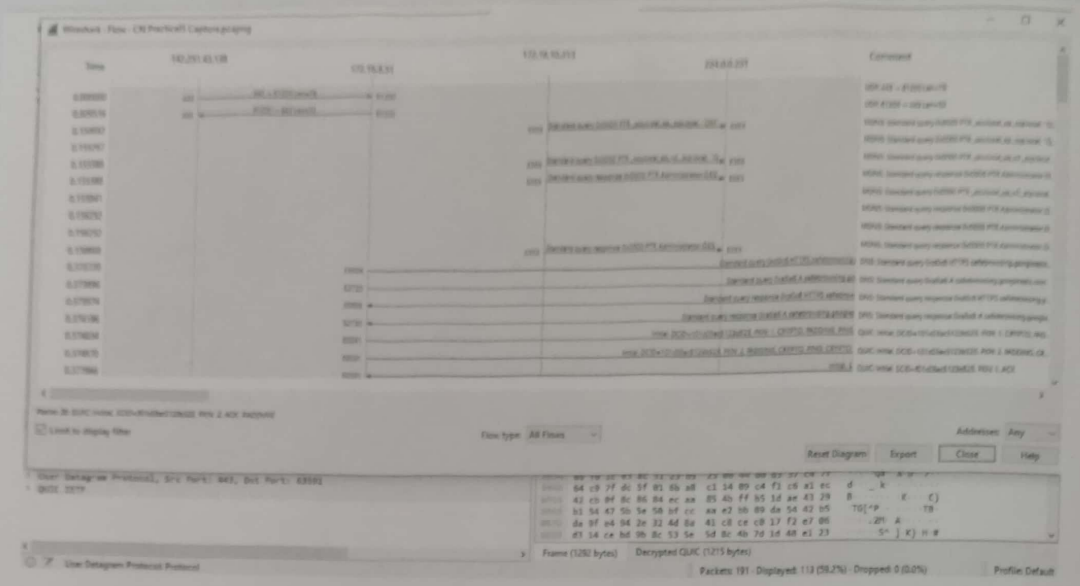
Enter : ip

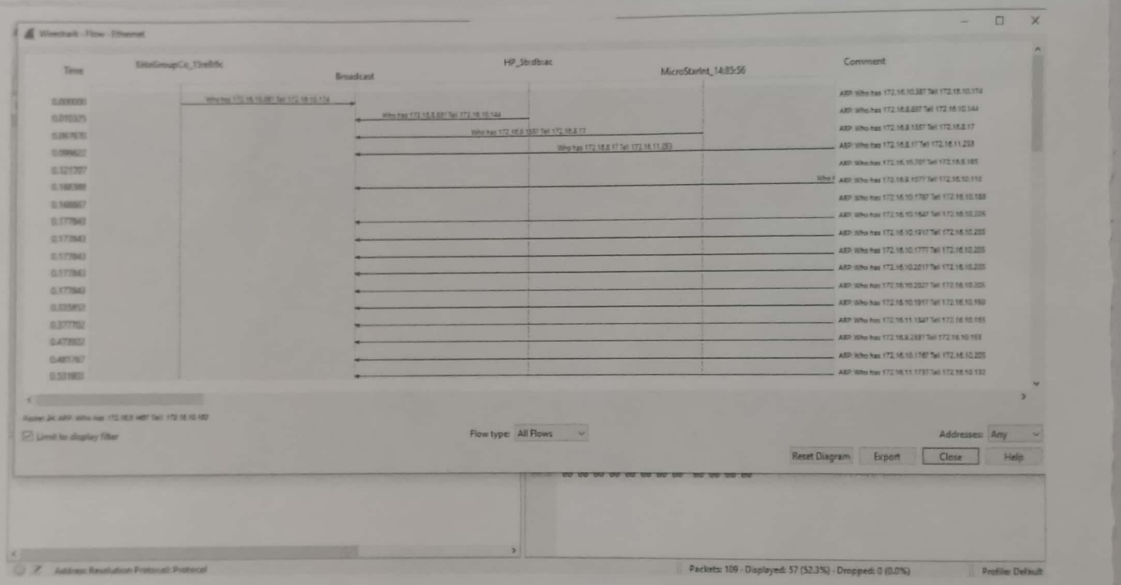
or

icmp

The selected packets can be inspected to observe the source IP address, destination IP address and ICMP information

Scanned with OKEN Scanner





Result

Thus, network packets were successfully captured and analyzed using Wireshark. Different protocol packets such as TCP, UDP, ARP, DNS, HTTP, IP/ICMP and DHCP were filtered and inspected.

Scanned with OKEN Scanner

Exp. No. 7

29.9.26

Flow Control at Data Link Layer

Aim:

Write a program to implement flow control at data link layer using sliding window Protocol. Simulate the flow of frames from one node to another.

Program:

Sender

```
import java.io.*;
import java.util.*;
public class Sender {
    public static void main(String[] args) throws
                Exception {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter Window Size: ");
        int windowSize = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Text Message: ");
        String message = sc.nextLine();
        int nextFrame = 0;
        while(nextFrame < message.length()) {
            int end = Math.min(nextFrame + windowSize,
                        message.length());
```

Scanned with OKEN Scanner

```
for (int i = nextFrame; i < end; i++) {
    String frame = "Frame No: " + i + ", DATA:
                   + message.charAt(i);
    System.out.println(frame);
    fw.write(frame);
    fw.write("\n");
}
fw.close();
BufferedReader br = new BufferedReader(new
    FileReader("Receiver_Buffer.txt"));
String response = br.readLine();
br.close();
if (response.startsWith("ACK")) {
    int ack = Integer.parseInt(response.
              replaceAll("[^0-9]", ""));
    System.out.println("ACK received: "+ack);
    nextFrame = ack;
}
System.out.println("All frames sent");
System.out.println("Message:"+message);
sc.close();
}
}
```

Receiver

```
import java.io.*;
public class Receiver {
    public static void main(String[] args) throws
                                    Exception {
        int expectedFrame = 0;
        System.out.println("Receiver started");
        while (true) {
            File senderFile = new File("Sender_Buffer.txt");
            if (!senderFile.exists()) {
                Thread.sleep(1000);
                continue;
            }
            if (senderFile.length() == 0) {
                Thread.sleep(1000);
                continue;
            }
            BufferedReader br = new BufferedReader(
                new FileReader("Sender_Buffer.txt"));
            String line;
            boolean error = false;
            int nackFrame = expectedFrame;
            System.out.println("Reading sender Buffer");
            FileWriter fw = new FileWriter("Receiver_Buffer
                                                .txt");
            if (error) {
                fw.write("NACK: " + nackFrame);
```

```
System.out.println("NACK sent: " + nackFrame
} else {
    fw.write("ACK: " + expectedFrame);
    System.out.println("ACK sent: " +
                       expectedFrame);
}
fw.close();
FileWriter clear = new FileWriter("sender_Buffer.tx
clear.close();
Thread.sleep(1000);
}
}
}
```

Result:

Thus a program to implement flow control at Data Link Layer using sliding window is executed successfully.

29/9/22

Exp No 8
30.9.26

NMAP to discover Live Hosts Using Nmap Scans on TryHackMe Platform

Aim:

To discover Live Hosts Using Nmap Scans (ARP, ICMP, TCP/UDP) on TryHackMe Platform

NMAP

Nmap stands for Network Mapper. It is an open-source network subnetting scanning tool used to identify live hosts & network services.

Send a packet with the following

From:

computer1

To

computer1

Packet Type:

arp_request

Data

computer6

From computer1

To computer1

Packet Type: ARP Request

Scanned with OKEN Scanner

1) How many devices can see the ARP Request?

4

2) Did computer6 receive the ARP Request?

nay

Send a packet with the following

From

computer 4

To

computer 4

PacketType

arp-request

Data

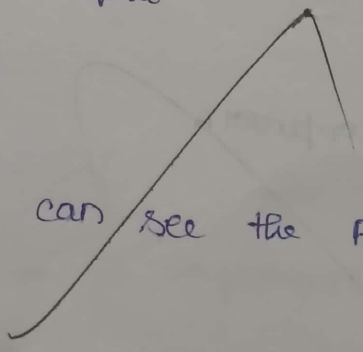
computer 6

From computer 4

To computer4

Packet Type: "ARP Request"

Data : computer 6



1) How many devices can see the ARP Request?

4

2) Did computer6 reply to ARP Request?

Scanned with OKEN Scanner

Send a packet with the following

From computer1

To computer3

Packet Type: "Ping Request"

1) What type of packet did computer1 send before the ping?

ARP Request

2) What type of packet did computer1 receive before it was able to send the ping?

ARP Response

3) How many computers responded to the ping request?

1

Result:

Thus to discover live Hosts using Nmap scans (ARP, ICMP, TCP/UDP) on TryHackMe Platform is executed successfully.

30/9/22

Exp No: 9
30.9.26

Implementation of Subnetting in Cisco Packet Tracer Simulator

Aim:

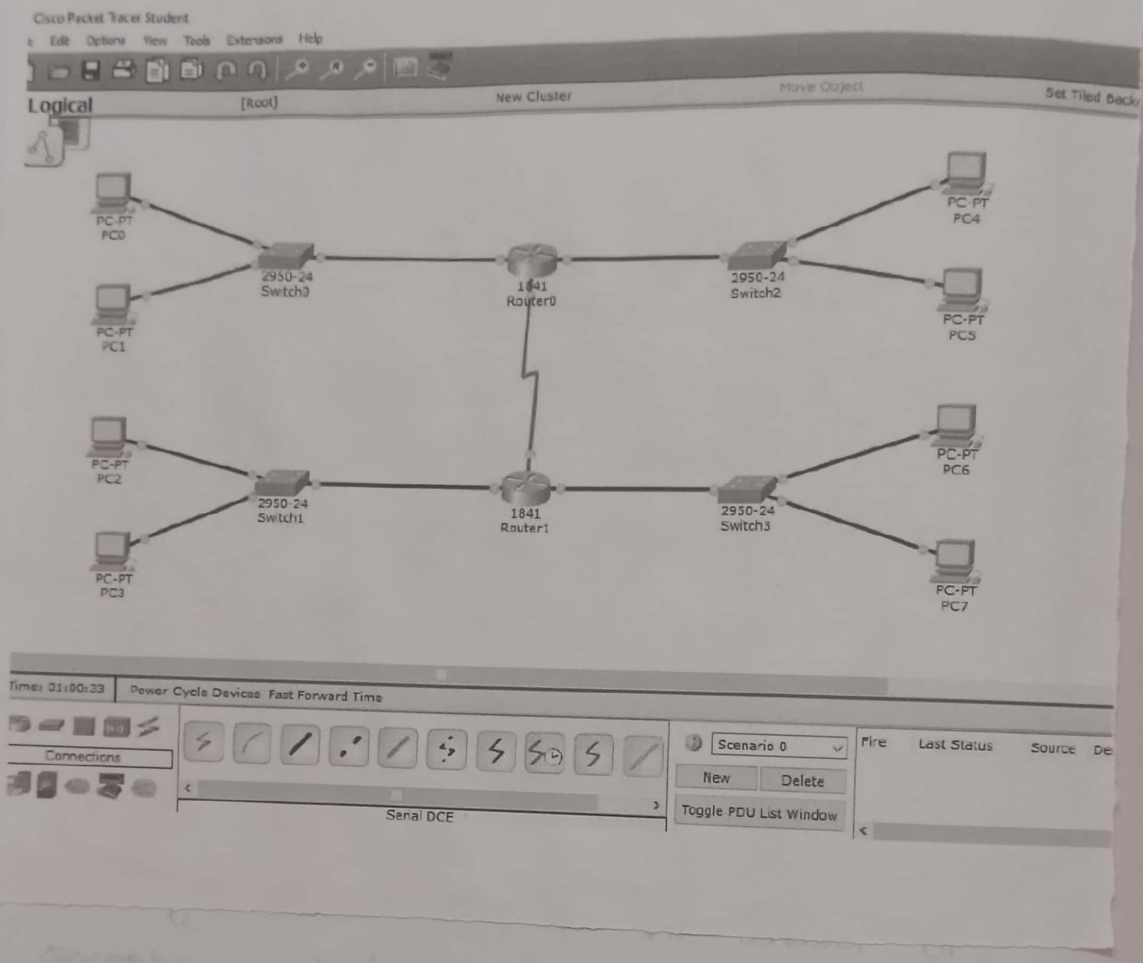
To implement Subnetting in Cisco Packet Tracer simulator.

Creating a network topology:

The first step in implementing classless IP subnetting is to create a network topology in Packet Tracer. To create a network topology in Packet Tracer, select the "New" button in the top left corner then select Network and Generic

Subnetting

To subnet the network address of 192.168.1.0 to provide enough space for at least 5 addresses for end devices, the switch and the router, we can use a /27 subnet mask. This will give us 8 subnets with 30 host addresses each.



Result:

Thus an experiment to implement Subnetting in Cisco Packet Tracer is executed successfully.

Scanned with OKEN Scanner

Ex. No: 10
30.9.26

End - End Communication at Transport Layer

Aim:

Implement echo client server using TCP/UDP sockets

1. TCP Echo Client - Server

TCP Server

```
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM
server.bind((HOST, PORT))
server.listen(1)
print("TCP server is waiting for connection...")
conn, addr = server.accept()
print("Connected by", addr)
while True:
    data = conn.recv(1024)
    if not data:
        break
    print("Client:", data.decode())
    conn.sendall(data)
conn.close()
server.close()
```

Scanned with OKEN Scanner

TCP client

```
import socket
HOST = '127.0.0.1'
PORT = 5000
client = socket.socket(socket.AF_INET, socket.SOCK_STRE
client.connect((HOST, PORT))
while True:
    message = input("Enter message:")
    if message.lower() == "exit":
        break
    client.send(message.encode())
    data = client.recv(1024)
    print("Echo from server:", data.decode())
client.close()
```

Output

Server

```
TCP Server is waiting for connection...
Connected by ('127.0.0.1', 52130)
Client: Hello
Client: Networking
```

Client

```
Enter message: Hello
Echo from server: Hello

Enter message: exit
```

Scanned with OKEN Scanner

2. UDP Echo Client - Server

UDP Server

```
import socket
HOST = '127.0.0.1'
PORT = 5001
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((HOST, PORT))
print("UDP Server is running...")
while True:
    data, addr = server.recvfrom(1024)
    print("Client", data.decode())
    server.sendto(data, addr)
```

UDP client

```
import socket
HOST = '127.0.0.1'
PORT = 5001
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
while True:
    message = input("Enter message:")
    if message.lower() == "exit":
        break
    client.sendto(message.encode(), (HOST, PORT))
    data, server = client.recvfrom(1024)
    print("Echo from server:", data.decode())
client.close()
```

Scanned with OKEN Scanner

Exp. No: 11
28-10-26

Output

UDP Server

UDP Server is running...

Client: Hello

Enter message: Hello

Echo from server: Hello

Enter message: exit

Result:

The Echo Client - Server application was successfully implemented using both TCP and UDP sockets.

Scanned with OKEN Scanner

Exp. No: 11

2.10.26

Implement ping program

Aim:

To implement your own ping program

Program:

```
import os
import platform
import subprocess

def ping(host="google.com", count=4):
    param = "-n" if platform.system().lower()
        == "windows" else "-c"
    command = ["ping", param, str(count), host]
    try:
        output = subprocess.check_output(command,
            universal_newlines=True)
        print(output)
    except Exception as e:
        print(f"Ping failed: {e}")

if __name__ == "__main__":
    ping("8.8.8.8")
```

Scanned with OKEN Scanner

Input:

Server Host : 127.0.0.1

Server Port : 12000

Number of Pings : 5

Output:

[SERVER] Listening on 127.0.0.1 : 12000

Reply from 127.0.0.1 : 12000 | Ping 1 | RTT = 0.18 ms

Reply from 127.0.0.1 : 12000 | Ping 2 | RTT = 0.10 ms

Reply from 127.0.0.1 : 12000 | Ping 3 | RTT = 0.09 ms

Result:

UDP ping communication was successfully established between the client and server and the RTT for each packet was measured.

Scanned with OKEN Scanner

Exp: No: 12

2.10.26

Write a code using RAW sockets to implement packet sniffing

Aim:

To write a program using RAW sockets to implement packet sniffing.

Program

```
import socket
import struct

sniffer = socket.socket( socket.AF_INET, socket.SOCK_RAW,
                         socket.IPPROTO_IP)

host = socket.gethostbyname( socket.gethostname())
sniffer.bind ((host, 0))
sniffer.setsockopt (socket.IPPROTO_IP, socket.IP_HDRINCL, 1)
sniffer.ioctl( socket.SIO_RCVALL, socket.RCVALL_ON )
print(" Packet Sniffer Started ...")
print (" Press Ctrl+C to stop.")
try:
    while True:
        raw_data, addr = sniffer.recvfrom (65535)
        ip_header = raw_data [0:20]
        iph = struct.unpack ( '    ', ip_header )
        source_ip = socket.inet_ntoa (iph [8])
        dustination_ip = socket.inet_ntoa (iph [9])
        protocol = iph [6]
```

```
        packet_length = iph[2]
        print("Source IP:", source_ip)
        print("Destination IP:", destination_ip)
        print("Protocol:", protocol)
        print("Packet Length:" packet_length)
except KeyboardInterrupt:
    print("Packet Sniffer stopped.")
finally:
    sniffer.ioctl(socket, SIO_RCVALL, socket.RCVALL_O
    sniffer.close()
```

Output

```
Packet Sniffer Started...
Press Ctrl+C to stop
Source IP : 192.168.1.5
Destination IP : 142.250.183.14
Protocol : 6
Packet Length : 60
```

Result

Thus a program to implement ~~RAW socket~~ packet sniffing using RAW socket is executed successfully.

Scanned with OKEN Scanner

5.10.26

Internetwork using Router

Aim

To design and configure a simple internetwork using routers in Cisco Packet Tracer.

Main Idea:

Different networks are created with 3-4 hosts each and connected using router. Static IP addresses are assigned to PCs and router interfaces. The communication between different networks is then tested.

Procedure:

1. Open Cisco Packet Tracer
2. Create different networks with PCs
3. Connect each network to a router using suitable cables.
4. Assign static IP addresses to all PCs
5. Configure IP addresses and subnet masks for the router interfaces.
6. Configure routes so that router knows how to reach the other networks.
7. Use the ping command to check connectivity btw PCs in different networks.
8. Use tracert to observe the path followed by packets.

Scanned with OKEN Scanner

9. Check the routing table using show ip route

Result:

The internetwork was successfully designed and configured in cisco packet Tracer, and communication between different networks was verified using ping/ Tracert.

Scanned with OKEN Scanner

Exp 13-b

Aim

To design and configure an internetwork using a wireless router, DHCP server and Internet cloud in cisco Packet Tracer.

Procedure

1. Create the network using

Wireless router

PC

Internet cloud

cisco.com server

2. Configure the wireless router and its wireless settings

3. Connect the wireless router to Internet cloud.

4. Configure the cloud's connection ports and select cable as the provider network

5. Configure the cisco.com server as a DHCP server.

6. Configure the cisco.com server as a DNS server

7. Configure the server's static IP as 208.67.222.220

8. On the PC, use DHCP to obtain its IP configuration automatically.

Scanned with OKEN Scanner

9. Use

ipconfig / release

ipconfig / renew

10. Test connectivity using

ping cisco.com.

Result

The wireless internetwork was successfully configured using a wireless router, DHCP server and Internet was verified using ping.

Scanned with OKEN Scanner

Exp: No: 14

10.10.26

RIP routing

Aim:

To design and configure a network using RIP (routing information protocol) in cisco packet tracer.

Main idea:

RIP is a dynamic routing protocol. It allows router to automatically exchange info about connected networks and select a route based on hop count.

Procedure:

1. Create Topology with
   - 2 pcs
   - 3 routers
   - serial/ Ethernet connections
2. Assign IP addresses to the router interfaces.
3. Enable the router interfaces using no shutdown.
4. Configure RIP on each router
5. Add the directly connected networks using the network command.
6. Test connectivity using ping command.

Scanned with OKEN Scanner

7. Use tracert to observe the route taken by packets

8. If one route fails, RIP automatically selects another available route.

RIP Commands

Router 0

router rip

network 10.0.0.0

network 192.168.1.252

network 192.168.1.252

Result

Thus a network using RIP in cisco packet tracer is created successfully.

Scanned with OKEN Scanner